

TASK プラズマ輸送ライブラリ化プロジェクト

日本語マニュアル・報告書

(tr / ti / wr / wrx / fp / eq / tot モジュール, Phase L-0 ～L-7)

k-yoshimi (東京大学)

2026 年 4 月 19 日

Abstract

本書は TASK コード群 (プラズマ統合輸送シミュレーション) のうち, 7 つの物理/オーケストレータモジュール—**tr** (1 次元輸送), **ti** (不純物輸送), **wr** (波動レイトレーシング), **wrx** (拡張レイトレーシング), **fp** (Fokker–Planck 解析), **eq** (MHD 平衡), **tot** (統合シミュレータ)—を共有ライブラリ化し, *Python* から利用可能にした一連の作業 (Phase L-0 ~L-7) を, Fortran や *ctypes* に馴染みのない初学者にも読めるようにまとめたマニュアルである.

本改訂 (2026-04-19) では, **eq** モジュール (第 8 章) と **tot** オーケストレータ (第 9 章) の使い方を新規追加し, 6 個揃った MCP サーバ群と **eq** の F90 モダナイゼーション (Phase F-1 ~F-5) も併せて反映した.

使い方章 (第 3~9 章) は**です・ます調**で平易に記述し, 技術章 (第 10 章以降) は**だ・である調**で手短にまとめる.

Contents

1	はじめに	5
1.1	本書の目的	5
1.2	読者対象 (いちばん大事なところ)	5
1.3	本書で何ができるようになるか	5
1.4	前提ソフトウェア	6
1.5	本書の構成	6
1.6	表記規約	6
2	共通アーキテクチャ	7
2.1	全体像—3 層アーキテクチャ	7
2.2	共有ライブラリ (.so) とは	8
2.3	5 関数 C ABI	8
2.4	エラーコード 0~4	8
2.5	5 関数のライフサイクル	9
2.6	PIC (位置独立コード) とは	9
2.7	Python ラッパの 2 層設計	10
2.8	ライブラリの探し方	10
2.9	RTLD_LAZY ロード	10
2.10	メモリレイアウト (C と Fortran の違い)	10
2.11	本章のまとめ	11
3	tr モジュール	12
3.1	概要—tr は何をする	12
3.2	前提とビルド	12
3.2.1	ライブラリをビルドする	12
3.2.2	エクスポートされている関数を確認する	13
3.2.3	Python から見えるようにする	13
3.3	最短 hello-world (5 行)	13
3.4	パラメータの指定方法	14
3.4.1	方法 A —スカラー (set_params)	14
3.4.2	方法 B —配列要素 (set_param)	14
3.4.3	方法 C —文字列パラメータ (set_param_str)	14
3.5	コンテキストマネージャ with の解説	14
3.6	FAQ / つまづきどころ	15
3.7	サポートされているパラメータ	15
3.8	TrState 出力	17
3.9	変更履歴—Phase L-0 ~L-7	17
3.10	テスト	17

4	ti モジュール	19
4.1	概要—ti は何をする	19
4.2	前提とビルド	19
4.3	最短 hello-world	19
4.4	パラメータ指定	20
4.5	TiState の主なフィールド	20
4.6	FAQ / つまづきどころ	20
4.7	入力パラメータ表 (抜粋)	21
4.8	変更履歴—Phase L-0 ～L-7	22
4.9	テスト実行	22
5	wr モジュール	23
5.1	概要—wr は何をする	23
5.2	前提とビルド	23
5.3	最短 hello-world	23
5.4	WrState の構造	24
5.5	FAQ	24
5.6	変更履歴	24
5.7	テスト	25
6	wrx モジュール	26
6.1	概要—wrx は何をする	26
6.2	前提とビルド	26
6.3	最短 hello-world	26
6.4	既知の制約 — wrx_run と libgrf	27
6.5	WrxState の構造	27
6.6	FAQ	27
6.7	変更履歴	27
6.8	テスト	27
7	fp モジュール	28
7.1	概要—fp は何をする	28
7.2	前提とビルド	28
7.3	最短 hello-world	28
7.4	FpState の主なフィールド	29
7.5	FAQ	29
7.6	入力パラメータ表 (抜粋)	29
7.7	変更履歴—Phase L-0 ～L-7	31
7.8	テスト	31
8	eq モジュール	32
8.1	概要—eq は何をする	32
8.2	前提とビルド	32
8.3	最短 hello-world	33
8.4	パラメータ指定—set_param と set_param_str	33
8.4.1	数値スカラー / 配列要素	33
8.4.2	文字列パラメータ—set_param_str	33
8.5	FAQ / つまづきどころ (EQ 固有)	33
8.6	入力パラメータ表 (抜粋, 約 60 エントリ)	34
8.7	EqState 出力	35
8.8	F90 モダナイゼーション (Phase F-1 ～F-5)	36
8.9	変更履歴—Phase L-0 ～L-7 と F-1 ～F-5	36

8.10 テスト	36
9 tot モジュール (オーケストレータ)	37
9.1 概要—tot は何をする	37
9.2 前提とビルド	37
9.3 最短 hello-world	37
9.4 名前空間プレフィックス設計	38
9.4.1 kwargs ではなく dict	38
9.4.2 文字列パラメータ	38
9.5 各モジュールへの delegation	38
9.6 既知の制約	39
9.7 変更履歴—Phase L-0 ～L-7	39
9.8 テスト	39
10 テストインフラ	40
10.1 4 層構成の意図	40
10.2 run_tests.sh と test_definitions.conf	40
10.3 Layer 1 —Equivalence テスト	41
10.4 Layer 2 —C ABI テスト	41
10.5 Layer 3 —Python ラッパ	41
10.6 Layer 4 —Sweep テスト	41
10.7 テストの走らせ方	41
11 作業状況・残作業	42
11.1 モジュール別完了マトリクス	42
11.2 eq F90 モダナイゼーション (F-1 ～F-5) 完了	42
11.3 MCP サーバ群 (6 個揃い)	43
11.4 Shim ポリシー (グラフィクス層)	43
11.5 F90 モダナイゼーション	43
11.6 その他の残作業	43
12 付録	45
12.1 Makefile ターゲット一覧	45
12.2 主な PR 一覧 (2026-04-19 改訂時点, #96 まで)	45
12.3 参考資料	46
12.4 用語集	46
12.5 連絡先・貢献	47
A 補足: Python 最小例カタログ	48
A.1 tr	48
A.2 ti	48
A.3 wr	48
A.4 wrx	48
A.5 fp	49
A.6 eq	49
A.7 tot	49
B 補足: C から直接呼ぶ例	50
C 補足: 回帰テストのテンプレート	51
D 補足: TikZ 依存関係グラフ	52

編集後記

53

Chapter 1

はじめに

この章で学ぶこと

本章では、本書が何であるかと、読者がどんな力を得られるかを整理します。ここは読み飛ばさずに目を通しておくと、後の章で迷子になりにくくなります。

1.1 本書の目的

TASK コード群 (京都大学・東京大学で長年開発されているプラズマ統合シミュレーションコード) の一部モジュールが、2026 年 4 月の集中開発で「共有ライブラリ (.so ファイル)」として配布・利用できるようになりました。これによって、

- Fortran の `tr2` や `fp` といった対話型プログラムを起動しなくても、
- Python スクリプトから関数呼び出し感覚でプラズマ輸送計算を回せる、

ようになりました。

本書はその使い方と作業内容を記録するマニュアル兼報告書です。

1.2 読者対象 (いちばん大事なところ)

次のような方を想定しています。

- プラズマ物理に興味がある学生、ただし Fortran は書いたことがない
- Python は少し触ったが、`ctypes` や共有ライブラリは聞いたことがない
- ターミナルで `cd`, `ls`, `make` くらいは使える

「C, Fortran, 共有ライブラリ」などの用語は最初に出てきた場所で毎回説明します。分からない用語が出てきたら、近くの [補足](#) や [FAQ](#) を確認してください。

1.3 本書で何ができるようになるか

読み終えたときに、次のことが自分の手で出来るようになります。

1. TASK のソースから `libtrapi.so` などの共有ライブラリをビルドする
2. Python スクリプト 5 行で `tr` / `ti` / `wr` / `wrx` / `fp` のいずれかの計算を回し、結果を `dict` として受け取る

3. パラメータ名と値を指定してパラメータスキャンループを書く
4. 4 層の回帰テスト (`run_tests.sh`) で自分の変更が Phase 0 ベースラインと $1e-10$ で一致することを確認する

1.4 前提ソフトウェア

- GNU/Linux (Ubuntu 22.04 以降で検証) もしくは同等 Unix
- gfortran 9 以降, gcc 9 以降, GNU Make
- Python 3.8 以降 (標準ライブラリのみで動作. NumPy は任意)
- Git

詳細な OS パッケージは `INSTALL_Ubuntu.md` を参照してください.

1.5 本書の構成

- 第 2 章—全モジュール共通の構造. TikZ 3 層図と 5 関数 API の説明.
- 第 3 ~7 章—`tr` / `ti` / `wr` / `wrx` / `fp` の各モジュール. 初学者向け hello-world と FAQ を含む.
- 第 8 章—`eq` (MHD 平衡) モジュール. 6 関数 C ABI (`eq_set_param_str` 含む) と PSIB の 0-origin 等の EQ 固有の癖を解説.
- 第 9 章—`tot` (統合) モジュール. 名前空間プレフィックス (`eq:RR`, `tr:DT` 等) で 6 モジュールを束ねる.
- 第 10 章—4 層テスト基盤.
- 第 11 章—作業状況 (完了/未完了, MCP サーバ群).
- 第 12 章—付録 (Makefile ターゲット, PR 一覧, 参考資料).

1.6 表記規約

- `$` はシェルのプロンプトです. `$` の後ろをターミナルに打ち込んでください.
- `>>>` は Python の対話プロンプトです.
- コード中の日本語コメントは, あえて全角で書くことで実際の挙動と区別しています. 実コードには入れないでください.

Chapter 2

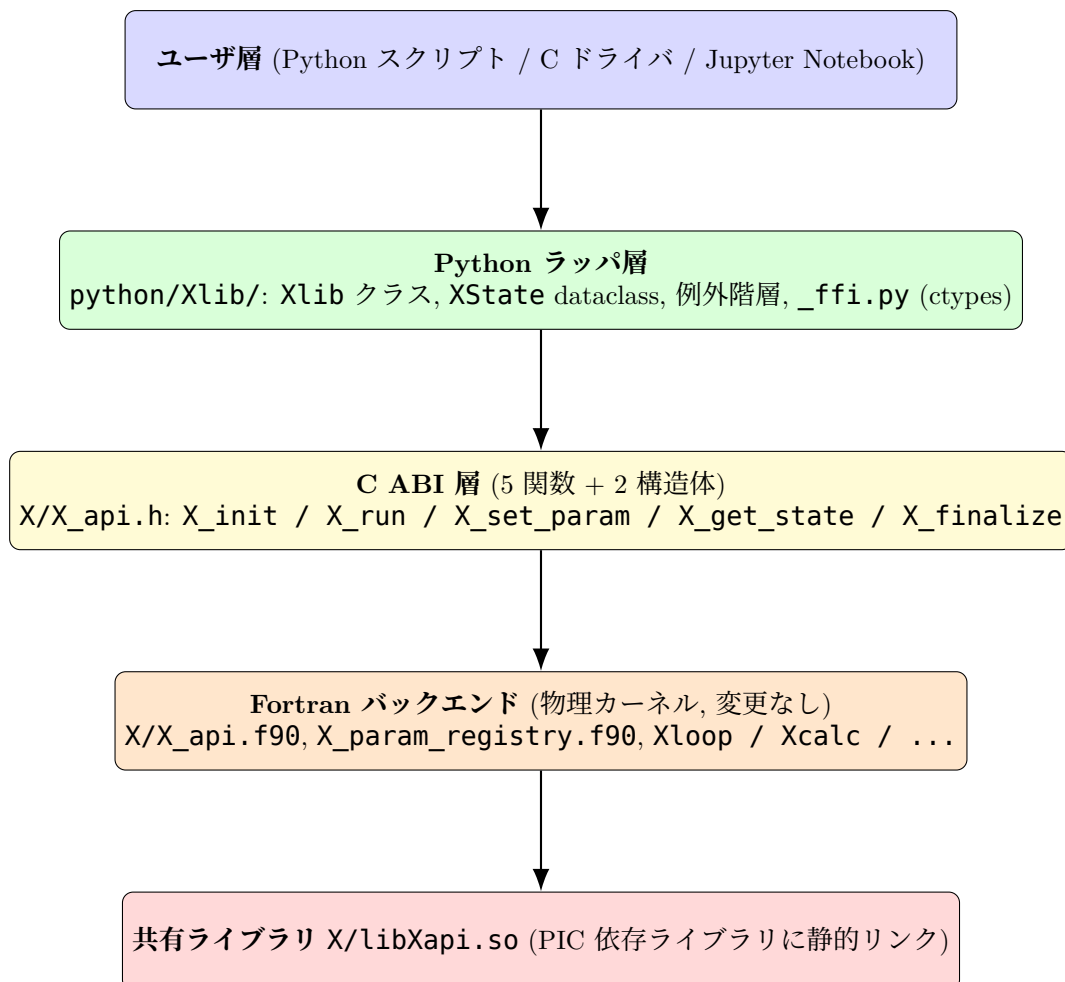
共通アーキテクチャ

この章で学ぶこと

本章では, `tr / ti / wr / wrx / fp` のすべてに共通する 3 層構造, 5 関数 C ABI, エラーコード 0~4, PIC ビルド, Python ctypes の 2 層設計を学びます. 以降の章はすべてこの章の応用です.

2.1 全体像—3 層アーキテクチャ

すべての Xlib (X は `tr / ti / wr / wrx / fp` のどれか) は同じ 3 層構造を持ちます.



各層の役割 (やさしい説明)

ユーザ層 あなたが書くコード. Python の関数呼び出しと同じ感覚で書けます.

Python ラップ層 Python から C の関数を呼び出すための「翻訳係」. 内部で `ctypes` を使って `.so` ファイルを読み込みます.

C ABI 層 C 言語の関数呼び出し規約 (Application Binary Interface) で Fortran と Python/C をつなぐ窓口. わずか 5 個の関数に絞ってあるので覚えやすいです.

Fortran バックエンド 物理計算の本体. 既存の `tr2` バイナリと全く同じソースコードを使うので, 数値が一致することが保証されます.

共有ライブラリ 上記全部をひとまとめにしたバイナリファイル. 拡張子 `.so` は “shared object” の略.

2.2 共有ライブラリ (.so) とは

補足

共有ライブラリ (`.so` ファイル) とは, 「プログラムが実行中に読み込んで使う, 再利用可能なコードのかたまり」のことです. Windows の `.dll`, macOS の `.dylib` に相当します.

通常のプログラムは自己完結した `.exe` で全機能を持ちますが, 共有ライブラリ方式では

- Python から `ctypes.CDLL("libtrapi.so")` と書けば読み込める
- C から `dlopen()`, コンパイル時リンクは `gcc -ltrapi` で OK

という具合にあとから好きな言語から呼べるのが利点です.

2.3 5 関数 C ABI

各モジュール `X` につき, エクスポートされる C 関数はたった 5 つです. これを覚えれば `X_` の頭文字さえ差し替えれば 5 モジュールすべてに共通です.

Listing 2.1: C ABI 5 関数 (`tr` の例. 他モジュールは `tr_` を `wr_` / `wrx_` / `ti_` / `fp_` に置換)

```

1 int tr_init(void);                               /* 初期化 */
2 int tr_run(int ntmax);                             /* 時間発展 */
3 int tr_set_param(const char* name, double value); /* パラメタ */
4 int tr_get_state(tr_state_t* state);               /* 状態取得 */
5 int tr_finalize(void);                             /* 後始末 */

```

いずれも返り値 `int` はエラーコード (次節参照) です.

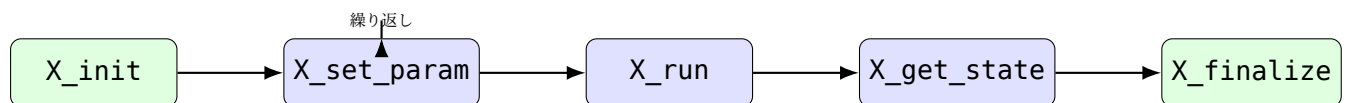
2.4 エラーコード 0~4

C ABI の返り値は次の 5 段階しかありません. 覚えやすいので暗記してしまってもかまいません.

コード	定数名	意味
0	X_OK	正常終了
1	X_ERR_INVALID	パラメータ名/値が不正
2	X_ERR_NOT_INIT	X_init を呼んでいない
3	X_ERR_CALC_FAILED	計算が失敗した
4	X_ERR_NOT_IMPL	未実装 (Phase L-2 時点のスタブ)

Python 側では、0 以外が返ると X libError の派生例外が投げられます。例: TrlibParamError, FplibNotInitError.

2.5 5 関数のライフサイクル



- X_init —1 プロセス 1 回だけ呼ぶ。Fortran 側の COMMON ブロックを初期化します。
- X_set_param —何度でも呼べます。例: "RR" (大半径), "BB" (磁場), "PN[1]" (1 番目のイオン密度) に値を与えます。
- X_run —指定したステップ数だけ時間発展を進めます。
- X_get_state —現在の状態を X_state_t 構造体 (C) に書き出します。Python では XState dataclass になります。
- X_finalize —片付け。通常は with 文が自動で呼んでくれます。

2.6 PIC (位置独立コード) とは

補足

PIC = Position Independent Code. 共有ライブラリ (.so) を作る時にどのアドレスにロードされても動くように gcc/gfortran が吐くコードのことです。通常の静的ライブラリ (.a) には要りません。

TASK では、旧来の .a アーカイブ (非 PIC) を保存したまま、PIC 版 lib*_pic.a を並行して生成するように各モジュールの Makefile を修正しました。これで tr2 バイナリのビルドは一切変わらず、共有ライブラリも作れるようになっています。

PIC 用の Makefile ターゲット

各モジュールには次のターゲットが追加されています。

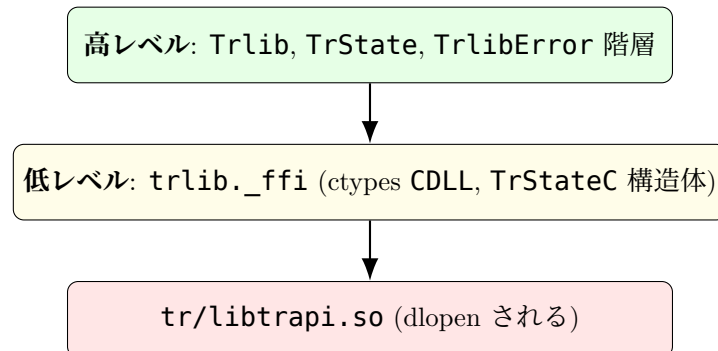
```

1 make -C lib libs_pic      # lib/ 配下の PIC アーカイブ
2 make -C pl  libs_pic      # pl/plcomm_pic.a など
3 make -C eq  libs_pic
4 make -C mt xp libs_pic
5 make -C bpsd libs_pic
6 make -C tr  libtrapi.so   # 最終成果物

```

2.7 Python ラッパの 2 層設計

Python 側はわざと 2 層に分けてあります.



低レベル `_ffi.py` は C ABI の鏡写しで, `ctypes.Structure` で `X_state_t` の各フィールドを同じ順序・同じ型で並べています. バイト単位で一致するので, Fortran が書き出した生データをそのまま Python 側で読めます.

高レベル `Xlib.py` は Python らしい API (`with` 文, `dataclass`, 例外クラス) を提供します.

2.8 ライブラリの探し方

`Xlib()` を引数なしで呼ぶと, 次の順で `.so` を探します.

1. 環境変数 `XLIB_PATH` (例: `TRLIB_PATH`, `FPLIB_PATH`)
2. < リポジトリ `root` > /X/libXapi.so (標準ビルド場所)
3. < リポジトリ `root` > /lib/libXapi.so (将来のインストール先)

明示的に指定したい場合は `Trlib(lib_path="/絶対/パス")` とします.

2.9 RTLD_LAZY ロード

補足

`ctypes.CDLL(..., mode=RTLD_LAZY)` は「実際にその関数が呼ばれる瞬間までシンボル解決を遅らせる」オプションです. TASK の共有ライブラリは, グラフィクス関連のシンボルが一部残ったまま作られるため, `RTLD_NOW` (即時解決) だとロードに失敗することがあります. `RTLD_LAZY` なら, 呼ばないパスの解決を後回しにできるので安全です.

2.10 メモリレイアウト (C と Fortran の違い)

注意

C は行優先, Fortran は列優先でデータを並べます. たとえば C の `double RN[NRMAX][NSMAX]` は Fortran の `REAL(KIND=8) :: RN(NSMAX, NRMAX)` とバイト単位で一致しますが, 添字の順序が逆です. Python 側の `XState dataclass` は `nrmx` 側を外側のリスト, `nsmax` を内側のリストとして見せるので, C 流の順番です.

2.11 本章のまとめ

これで第 3 章以降は、「API は 5 つ, エラーは 0~4, 例外は X `libError` 派生, with で書く」という 1 つのパターンを 5 回繰り返すだけで読み進められます.

Chapter 3

tr モジュール

この章で学ぶこと

本章では, TASK/TR (1 次元トカマク輸送シミュレーション) を Python から呼び出すライブラリ `trlib` の使い方を学びます. 共有ライブラリのビルド, 最短 hello-world (5 行), パラメータの与え方, FAQ, そしてテストを実行する方法までカバーします.

3.1 概要—tr は何をする

TASK/TR は, トカマクや球状トカマクの **1 次元 (半径方向) 輸送シミュレーション**を行うモジュールです. 「プラズマの半径方向の分布が, 時間とともにどう変化するか」を, 粒子数・温度・電流・磁場の平衡を解きながら追跡します.

得られる代表的な量は

- `RN[i][j]` —半径点 i での j 番目の粒子種の密度
 - `RT[i][j]` —同じく温度
 - `AJ[i]` —電流密度プロファイル
 - `QP[i]` —安全係数 q プロファイル
 - スカラー: `T` (時間), `WPT` (蓄積エネルギー), `Q0` (中心 q), `BETAN` (β_N) など 13 種
- です.

3.2 前提とビルド

3.2.1 ライブラリをビルドする

Listing 3.1: tr モジュールのライブラリビルド

```
1 $ cd /home/you/task           # TASK リポジトリのルートに移動
2 $ make -C lib libs_pic        # 下位ライブラリの PIC 版を作る
3 $ make -C pl libs_pic         # プラズマ共通モジュール
4 $ make -C eq libs_pic         # 平衡モジュール
5 $ make -C mt xp libs_pic      # 疎行列ソルバ
6 $ make -C bpsd libs_pic       # BPSD データ橋渡し
7 $ make -C tr libtrapi.so      # tr モジュールの共有ライブラリ本体
```

最後に `tr/libtrapi.so` というファイルが出来ていれば成功です.

```

1 $ ls -lh tr/libtrapi.so
2 -rwxr-xr-x 1 user user 8.2M 4月 18 23:00 tr/libtrapi.so

```

ファイルサイズは環境によって違いますが、だいたい 5 ～10 MB くらいです。

3.2.2 エクスポートされている関数を確認する

「本当に 5 つの関数が入っているか」は nm コマンドで確認できます。

```

1 $ nm -D tr/libtrapi.so | grep ' T tr_'
2 000000000005a1b0 T tr_finalize
3 000000000005a090 T tr_get_state
4 0000000000059e10 T tr_init
5 0000000000059f80 T tr_run
6 0000000000059d30 T tr_set_param
7 0000000000059ca0 T tr_set_param_str

```

T 列のシンボルが「エクスポートされた関数」を示します。

3.2.3 Python から見えるようにする

```

1 $ export PYTHONPATH=/home/you/task/python:$PYTHONPATH

```

こうすると import trlib が通ります。

3.3 最短 hello-world (5 行)

Listing 3.2: trlib 最小例

```

1 from trlib import Trlib          # (1) インポート
2 with Trlib() as tr:              # (2) 初期化 (= tr_init 自動)
3     tr.set_params(RR=3.0, BB=3.0) # (3) 大半径 3m, 磁場 3T
4     tr.run(ntmax=10)              # (4) 10 ステップ時間発展
5     state = tr.get_state()        # (5) 状態を取得 (型は TrState)
6 print(state.scalars["T"])        # 最終時刻 (秒)

```

行ごとの解説

(1) from trlib import Trlib

パッケージ trlib からクラス Trlib を読み込みます。Trlib は共有ライブラリのハンドル (持ち手) です。

(2) with Trlib() as tr:

with 文は Python のコンテキストマネージャ機能です。入る時に `__enter__` (= `tr_init`), ブロックを抜けるときに `__exit__` (= `tr_finalize`) が自動で呼ばれます。後片付けを忘れる心配がありません。

(3) tr.set_params(RR=3.0, BB=3.0)

set_params はスカラー値を一括でセットするヘルパーです。Fortran の /TR/ ネームリストと同じ名前 (RR = 大半径, BB = トロイダル磁場) をキーワード引数で指定します。

(4) tr.run(ntmax=10)

時間発展を 10 ステップ進めます。ステップ幅は DT (デフォルト 0.01 秒) です。

(5) state = tr.get_state()

現在の状態を TrState データクラスに写し取ります。state.scalars["T"] で現在時刻, state.RT[i][j] で温度プロファイルが取れます。

期待される出力例

```

1 $ PYTHONPATH=python python3 examples/quickstart.py
2 NT=50  NRMAX=50  NSMAX=2
3 T      = 0.5
4 WPT    = 8.3e+05
5 Q0     = 0.96
6 BETAA= 0.42

```

3.4 パラメータの指定方法

3通りの指定方法があります.

3.4.1 方法 A —スカラー (set_params)

Python のキーワード引数で一括指定できます.

```

1 tr.set_params(RR=7.5, RA=2.0, BB=5.3, DT=0.05, NTMAX=200)

```

3.4.2 方法 B —配列要素 (set_param)

キーワード引数には [] が使えないので, 配列要素は set_param を使います. 1-origin (1 始まり) です.

```

1 tr.set_param("PN[1]", 1.0)    # 1 番目の粒子種の密度
2 tr.set_param("PN[2]", 1.0)    # 2 番目
3 tr.set_param("CDW[12]", 0.5)  # 配列 CDW の 12 番目

```

3.4.3 方法 C —文字列パラメータ (set_param_str)

TR には平衡データファイル名 KNAMEQ など文字列パラメータがあります. これは専用 API で与えます.

```

1 tr.set_param_str("KNAMEQ", "eqdata.ITER01")

```

3.5 コンテキストマネージャ with の解説

補足

Python の with 文は「ブロックに入るときと抜けるときに必ず後片付けを走らせる」仕組みです. ファイル操作の with open(...) as f: と同じ発想です. Trlib も __enter__ / __exit__ を実装してあるので, 例外が発生しても tr_finalize が必ず呼ばれ, メモリやグローバル状態が掃除されます.

明示的に書くなら次と同じ意味です:

```

1 tr = Trlib()
2 try:
3     tr.set_params(RR=3.0)
4     tr.run(ntmax=10)
5     state = tr.get_state()
6 finally:
7     tr.close()

```


3.6 FAQ / つまづきどころ

Q1. `FileNotFoundError: libtrapi.so not found ...` が出る

まだビルドしていないか、場所が違います。次を試してください:

1. `ls tr/libtrapi.so` が存在するか確認
2. 無ければ `make -C tr libtrapi.so` を実行
3. どうしても別の場所に置きたければ `export`
`TRLIB_PATH=/path/to/libtrapi.so`

Q2. `TrlibParamError: ierr=1` が出る

存在しないパラメータ名を指定した、または配列の添字が範囲外です。`tr/tr_param_registry.f90` の `SELECT CASE` 節に登録されているかを確認してください。追加するには Fortran 側に 1 行足すだけです (再ビルドが必要)。

Q3. `set_params(PN__1=1.0)` と書いてしまった

`__` (アンダースコア 2 つ) を含むキーは、配列構文の書き間違いとみなして明示的にエラーを出します。正しくは `tr.set_param("PN[1]", 1.0)` です。

Q4. 同じプロセスで 2 個の `Trlib()` を作れる?

作れますが**意味がありません**。Fortran 側は `COMMON` ブロックでグローバル状態を持つので、2 個目の `tr_init` は最初の状態を上書きします。プロセス分離 (multiprocessing) を使ってください。

Q5. 結果が `tr2` (CLI 版) と一致しない

回帰テスト `trlib_equivalence` を実行して Phase 0 ベースラインとの差分を確認してください。

```
1 bash test_run/run_tests.sh trlib_equivalence
```

`tol` は `1e-10`。許容値以上ずれる場合は、おそらく登録漏れのパラメータを指定していません。

Q6. NumPy が必要?

必要ありません。`TrState` は Python の `list` です。NumPy を使いたい場合は `import numpy as np; np.array(state.RT)` で変換できます。

3.7 サポートされているパラメータ

`tr/tr_param_registry.f90` に登録されている主なパラメータを表 3.1 に示します。名前は Fortran の `/TR/` ネームリストと一致しています。

Table 3.1: tr モジュールの主な登録パラメータ

名前	型	意味
RR	double	プラズマ大半径 [m]
RA	double	小半径 [m]
RKAP	double	elongation (縦横比)
RDLT	double	triangularity
BB	double	トロイダル磁場 [T]
PHIA	double	全磁束 [Wb]
RIPS	double	開始時刻のプラズマ電流 [MA]
RIPE	double	終了時刻のプラズマ電流 [MA]
MODELG	int	幾何モデル (1: 解析, 3:eqdata)
NSMAX	int	粒子種の数
PA[i]	double 配列	質量数 (i 番目の粒子種)
PZ[i]	double 配列	電荷
PN[i]	double 配列	中心密度
PNS[i]	double 配列	縁密度
PT[i]	double 配列	中心温度
PTS[i]	double 配列	縁温度
DT	double	時間ステップ [s]
NTMAX	int	時間ステップ数
NTSTEP	int	出力間引き
EPSLTR	double	反復収束判定
LMAXTR	int	最大反復数
MDLKAI	int	輸送モデル (カイ) 選択
MDLETA	int	抵抗モデル
MDLAD	int	輸送項切替
CDW[i]	double 配列	輸送係数 (12 要素)
CHP, CK0, CK1	double	カイ補正係数
MDLNB, MDLEC, MDLLH, MDLIC	int	NBI/EC/LH/IC 加熱モデル
MDLJBS	int	ブートストラップ電流モデル
MDLPEL, MDLST, MDLNF, MDLUF	int	ペレット/ソース/核融合/UFILE
PROFN1, PROFN2	double	密度プロファイル形状
PNC, MDLIMP	—	不純物混入
PNBR0, PNBW, PNBENG, PNBRTG	double	NBI 位置/幅/エネルギー
PICCD, PICR0, PICRW, PICNPR	double	ICRF パラメータ
PECCD, PECR0, PECRW, PECNPR	double	ECRF
PLHCD, PLHR0, PLHRW, PLHNPR, PLHTOT	double	LH
NGTSTP, NGRSTP	int	グラフ出力間引き
KNAMEQ	文字列	平衡データファイル名 (set_param_str)

3.8 TrState 出力

`tr.get_state()` は `TrState` dataclass を返します. フィールドは `tr/tr_api.h` の C 構造体 `tr_state_t` と対応します.

```

1 state.nt          # 時間ステップ数
2 state.nrmax       # 実働の半径点数 (<= 500)
3 state.nsmax       # 実働の粒子種数 (<= 8)
4 state.scalars["T"] # 時間 (秒)
5 state.scalars["WPT"] # 蓄積エネルギー
6 state.scalars["Q0"] # 中心 q
7 state.scalars["BETAN"] # beta_N
8 state.RN[0][0]    # RN[半径=0][粒子=0]
9 state.RT          # [nrmax][nsmax] の 2 次元リスト
10 state.AJ[0]       # 電流密度 (先頭)
11 state.QP[-1]      # 端 q
12 state.to_dict()   # JSON 化用 dict

```

3.9 変更履歴—Phase L-0 ~L-7

Phase	PR	日付	内容
L-0	#2	2026-04-18	Phase 0 回帰テスト基盤: <code>tr_iter01</code> , <code>tr_m0904</code> , <code>tr_tst2</code> 3 ケースの JSON ベースライン
L-1	#21	2026-04-18	<code>tr/Makefile</code> のグラフィクス分離. <code>tr2</code> はそのまま, <code>libtrapi.so</code> はグラフィクスフリー
L-2	#27	2026-04-18	C ABI 基盤 (<code>tr_api.h</code> , 5 関数スタブ <code>ierr=4</code>)
L-3	#33	2026-04-18	パラメータレジストリ (<code>tr_param_registry.f90</code> 38 ケース) および <code>tr_init/run/get_state/finalize</code> 実装
L-4	#35	2026-04-18	<code>make -C tr libtrapi.so</code> で <code>.so</code> が生成. <code>lib*_pic.a</code> を各所に追加
L-5	#44	2026-04-18	Python ラッパ <code>python/trlib/</code> : <code>Trlib</code> , <code>TrState</code> , 例外階層, <code>_ffi</code>
L-6	#53	2026-04-18	4 層テスト: <code>trlib_equivalence</code> , <code>_c_abi</code> , <code>_ffi</code> , <code>_wrapper</code> , <code>_sweep</code>
L-7	#—	2026-04-18	ドキュメント: <code>python/trlib/README.md</code> , <code>examples/*.py</code> , <code>architecture.md</code>

3.10 テスト

4 層の回帰テストを用意しています (第 10 章参照).

```

1 # 単一層
2 bash test_run/run_tests.sh trlib_equivalence # Layer 1
3 bash test_run/run_tests.sh trlib_c_abi       # Layer 2
4 bash test_run/run_tests.sh trlib_ffi         # Layer 3 (低レベル)
5 bash test_run/run_tests.sh trlib_wrapper     # Layer 3 (高レベル)
6 bash test_run/run_tests.sh trlib_sweep       # Layer 4

```

Phase L-5/L-6 での既知の問題: `tr_m0904` ベースラインは $\sim 10^{-8}$ の僅かなドリフトがあり, ベースライン側で pin しています. `tr_iter01` と `tr_tst2` は $1e-10$ で一致します.

Chapter 4

ti モジュール

この章で学ぶこと

本章では, TASK/TI (不純物輸送モジュール, 多価電荷イオンの輸送と原子過程をあわせて解く) を Python から呼ぶ `tilib` の使い方を学びます. TR と非常に似た API 体系です.

4.1 概要—ti は何をする

TASK/TI は, TR が扱わない不純物 (高 Z イオン) の輸送と, 各電荷状態間の電離・再結合を解きます. たとえばアルゴン ($Z = 18$) やタングステン ($Z = 74$) が壁から入ってきてどう分布するかをシミュレーションします.

4.2 前提とビルド

TR と同じ手順です.

```
1 $ cd /home/you/task
2 $ make -C lib libs_pic
3 $ make -C pl libs_pic
4 $ make -C eq libs_pic
5 $ make -C adpost libs_pic # 原子過程データ
6 $ make -C open-adas/adf11/adf11-lib libs_pic
7 $ make -C mt xp libs_pic
8 $ make -C bpsd libs_pic
9 $ make -C ti libtiapi.so
```

注意

`libtiapi.so` のトップレベル make ターゲットは執筆時点で develop にマージされておらず, フィーチャブランチ `feature/ti-library-L4-shared-lib` に存在します. マージ待ちの状態でも Python ラッパ側はライブラリが無ければ FFI テストをスキップするので, 他の章の学習は進められます.

4.3 最短 hello-world

```
1 from tilib import TiLib # クラス名は TiLib (大文字 L)
2 with TiLib() as ti:
3     ti.set_params(RR=6.2, BB=5.3, NSMAX=2)
4     ti.set_param("PN[1]", 0.7)
```

```

5     ti.run(ntmax=10)
6     state = ti.get_state()
7     print("T =", state.T)           # ti は T を dataclass 属性で持つ
8     print("RTA[0] =", state.RTA[0]) # 温度プロファイル

```

TR との違い

- クラス名が TiLib (大文字 L). エイリアス Tilib もエクスポート済み.
- スカラー T は state.scalars でなく state.T のように直接属性アクセスできます.
- 収束診断用に residual_loop_max, icount_loop_max, icount_mat_max が state 直下にあります.
- 密度は state.RNA, 温度は state.RTA, 速度は state.RUA (それぞれ [nrmax][nsa_max]).

4.4 パラメータ指定

TR と同じ要領です. 2 次元配列は "NAME[i,j]" でも指定可能です.

```

1 ti.set_param("PA[3]", 40.0)           # 3 番目の粒子種の質量数
2 ti.set_param("MODEL_BND[1,3]", 1.0)   # 2 次元配列
3 ti.set_param("BND_VALUE[1,3]", 1.0e-2)

```

補足

ti モジュールの /TI/ ネームリストは, 歴史的な経緯で PM(質量) のキーが plcomm の PA に pm=>pa でリネームされていました. Phase L-3 以降は PA を正式な API 名として採用し, PM は受け付けません. Argon の fixture は "PA[3]" = 40.0, "NPA[3]" = 18 で指定してください.

4.5 TiState の主なフィールド

```

1 state.nt, state.nrmax, state.nsa_max, state.nsmx
2 state.T           # 時間
3 state.residual_loop_max # 収束診断
4 state.icount_loop_max, state.icount_mat_max
5 state.RNA         # [nrmax][nsa_max] 密度
6 state.RTA         # [nrmax][nsa_max] 温度
7 state.RUA         # [nrmax][nsa_max] 速度
8 state.RBP, state.RQP # [nrmax] 磁場, 安全係数
9 state.RJP, state.ZEFF # [nrmax] 電流, Z_eff
10 state.BETA, state.BETAP # [nrmax] beta, beta_p

```

4.6 FAQ / つまづきどころ

Q. PM[3] を指定したがエラー

上記 note 参照. PA[3] を使ってください.

//

Q. DN0

Phase L-3 時点では登録されていません. Fortran 側 `ti_param_registry.f90` に `CASE("DN0")` を 1 行追加すれば拡張できます (再ビルド要).

Q. ti_ar のベースライン比較が微妙にずれる

`ti_ar` は `tr_m0904` と同様の軽微ドリフトがあり, ベースライン側で pin しています. ベースラインを再生成する場合は, `tools/extract_ti_metrics.py` でダンプ後に diff してください.

4.7 入力パラメータ表 (抜粋)

Table 4.1: ti モジュールの主な登録パラメータ

名前	型	意味
RR, RA, RKAP, RDLT, BB, RIP	double	装置幾何
NSMAX	int	粒子種の数
PA[i], PZ[i], PN[i], PNS[i], PT[i], PTPR[i], PTPP[i], PTS[i], PU[i], PUS[i]	double 1D	粒子種ごとの物理量
NPA[i], ID_NS[i]	int 1D	原子番号, ID
PROFN1[i], PROFN2[i], PROFT1[i], PROFT2[i], PROFU1[i], PROFU2[i]	double 1D	プロファイル形状
DT, NRMAX, NTMAX, NTSTEP, NGTSTEP, NGRSTEP	数値	時間・メッシュ
MAXLOOP, EPSLOOP, EPSMAT, MATTYPE	数値	非線形反復
MODEL_EQB, MODEL_EQN, MODEL_EQT, MODEL_EQU	int	平衡・物理量モデル

Table 4.1: ti モジュールの主な登録パラメータ

名前	型	意味
MODEL_KAI, MODEL_DRR, MODEL_VR	int	輸送モデル
MODEL_NC, MODEL_NF, MODEL_NB, MODEL_EC, MODEL_LH, MODEL_IC	int	各加熱機構
MODEL_CD, MODEL_SYNC, MODEL_PEL, MODEL_PSC	int	電流駆動/シンクロトロン/ペレット/源
NZMIN_NS[i], NZMAX_NS[i], NZINI_NS[i]	int 1D	電荷状態の最小/最大/初期
MODEL_BND[i, j], 2D BND_VALUE[i, j]		境界条件
PROFJ1, PROFJ2	double	電流プロファイル形状

4.8 変更履歴—Phase L-0 ～L-7

Phase	PR	日付	内容
L-0	#12	2026-04-18	ti 用 回 帰 テ ス ト 基 盤 (ti_min , ti_ar , ti_w)
L-1	#22	2026-04-18	ti/Makefile CORE / GRAPHICS / MENU 分割
L-2	#28	2026-04-18	ti_api.h + スタブ, tests/c_abi/
L-3	#34	2026-04-18	パラメータレジストリ (35 ケース, 1D/2D) と実装
L-4	#41	2026-04-18	libtiapi.so (フィーチャブランチ staging)
L-5	#47	2026-04-18	Python ラッパ python/tilib/
L-6	#57	2026-04-18	4 層テスト tilib_*
L-7	#64	2026-04-18	ドキュメント整備

4.9 テスト実行

```

1 bash test_run/run_tests.sh tilib_equivalence tilib_wrapper \
2   tilib_ffi tilib_c_abi tilib_sweep

```


Chapter 5

wr モジュール

この章で学ぶこと

本章では, TASK/WR (波動レイトレーシング: EC/LH 波の軌道追跡とパワー吸収) を Python から呼ぶ `wrlib` の使い方を学びます. 出力は「レイごとのピーク位置」や「半径プロファイル」といった独特の形になります.

5.1 概要—wr は何をする

TASK/WR は, 電子サイクロトロン (EC) 波や低域混成 (LH) 波などの高周波がプラズマ中でどう伝わり, どこで吸収されるかを幾何光学的レイトレーシングで追跡します. 結果としてレイごとの終端 (`rays_end`) や, 半径方向のパワー堆積プロファイル (`pos_nrs`, `pwr_nrs`) が得られます.

5.2 前提とビルド

```
1 $ cd /home/you/task
2 $ make -C lib libs_pic
3 $ make -C pl libs_pic
4 $ make -C eq libs_pic
5 $ make -C dp libs_pic # 分散関係モジュール (必要)
6 $ make -C mtxp libs_pic
7 $ make -C bpsd libs_pic
8 $ make -C wr libwrapi.so
```

5.3 最短 hello-world

```
1 from wrlib import Wrlib
2 with Wrlib() as wr:
3     wr.set_params(RR=3.0, BB=3.5, NSMAX=2)
4     wr.set_param("PN[1]", 0.7)
5     wr.run(nray_request=0) # 0: ネームリストの NRAYMAX を使う
6     state = wr.get_state()
7     print("pwrmax_rs =", state.scalars["pwrmax_rs"])
8     print("rays_end[0] =", state.rays_end[0])
```

補足

`nray_request` は `wr_run` の引数で、レイ本数の上書き用です。0 を渡すとネームリスト (NRAYMAX) の値をそのまま使います。

5.4 WrState の構造

3 つの実行時次元を持ちます。

- `nraymax` —レイの数
- `nrsmax` —半径 (rs) プロファイルの点数
- `nrlmax` —半径 (rl) プロファイルの点数

```

1 state.scalars["pos_pwrmax_rs"] # rs 軸での全体ピーク位置
2 state.scalars["pwrmax_rs"]   # そのパワー
3 state.nstp_end[i]             # レイ i の終了ステップ
4 state.rays_end[i]             # [9] 要素 (RAYS(0..8, i))
5 state.pos_nrs[k], state.pwr_nrs[k]
6 state.pos_nrl[k], state.pwr_nrl[k]
```

`rays_end[i]` は実行時 `nraymax` 本数にかかわらず常に長さ 9 (NEQ+1) です。

5.5 FAQ

Q. レイが全く動かない

ANGTIN / ANGPHIN / RFIN の初期条件が適切か確認してください。特に ANGPHIN=0 だと軸対称でレイが磁軸に沿ってしまうことがあります。

Q. `rays_end[i]` の 9 要素は何?

RAYS(0:NEQ, i) に対応し, 0: 位置, 1: 波数, 2..: 状態変数等の順です。詳細は `wr/wrcomm.f90` の定義をご覧ください。

5.6 変更履歴

Phase	PR	日付	内容
L-0	#-	2026-04-18	<code>wr_iter_lhcd</code> , <code>wr_test001</code> , <code>wr_tst2_ec</code> ベースライン
L-1	#-	2026-04-18	Makefile 分割
L-2	#-	2026-04-18	C ABI スタブ + <code>wr_api.h</code>
L-3	#-	2026-04-18	パラメータレジストリ
L-4	#-	2026-04-18	<code>libwrapi.so</code>
L-5	#-	2026-04-18	Python ラッパ <code>python/wrlib/</code>
L-6	#-	2026-04-18	4 層テスト <code>wrlib_*</code>
L-7	#-	2026-04-18	ドキュメント

5.7 テスト

```
1 bash test_run/run_tests.sh wrlib_equivalence wrlib_c_abi \  
2 wrlib_ffi wrlib_wrapper wrlib_sweep
```

Chapter 6

wrx モジュール

この章で学ぶこと

本章では, TASK/WRX (拡張レイトレーシング, 粒子種ごとのパワー分解が可能) を Python から呼ぶ `wrxlib` の使い方を学びます. `wrlib` と姉妹パッケージで, 同時に `import` できます.

6.1 概要—wrx は何をする

TASK/WRX は WR を拡張し, 吸収パワーを粒子種ごとに分解したり, より詳細な O/X モード変換を扱ったりします. 出力は WR と似ていますが, `pwr_nsa` (粒子種ごとのパワー), `pwr_nsa_nray[i][j]` (レイ `i`, 種 `j` のパワー) という 2 次元量が加わります.

6.2 前提とビルド

```
1 $ make -C lib libs_pic
2 $ make -C pl libs_pic
3 $ make -C eq libs_pic
4 $ make -C dp libs_pic
5 $ make -C mt xp libs_pic
6 $ make -C bpsd libs_pic
7 $ make -C wrx libwrxapi.so
```

6.3 最短 hello-world

```
1 from wrxlib import Wrxlib
2 with Wrxlib() as wrx:
3     wrx.set_params(MODELG=2, RR=6.2, BB=5.3, NSMAX=2, NRAYMAX=1)
4     wrx.set_param("RFIN[1]", 170.0e3)      # 170 GHz
5     wrx.run(nstpmax=0)                      # wr_run と同じく 0 で既定
6     state = wrx.get_state()
7     print("pwr_tot =", state.scalars["pwr_tot"])
8     print("per-species pwrmax_rs =", state.pwrmax_rs_nsa)
```

6.4 既知の制約 — wrx_run と libgrf

注意

L-4 時点の libwrxapi.so は, wrcalpwr.f90 から libgrf::grd1d への動的依存が残っています. 共有ライブラリ経由で wrx_run を呼ぶとセグメンテーション違反を起こす可能性があります.

回避策:

- テストは WRX_RUN_OK=1 環境変数でのみ有効化 (TestWrxlibRun).
- フルの wrx_run 相当は Layer 1 (wrx/wrxregress) か静的リンクの Layer 2 C ハーネスを使う.

6.5 WrxState の構造

```

1 state.nraymax, state.nstpmax, state.nsamax, state.nsmay
2 state.modelg, state.mdlwrq
3 state.scalars["pwr_tot"]
4 state.nstp_end[i]                # レイ終了ステップ (別名 nstpmax_nray)
5 state.pwr_nray[i]                # レイごとの総吸収パワー
6 state.pwr_nsa[j]                # 種ごとの総吸収パワー
7 state.pwr_nsa_nray[i][j]        # [レイ][種] の 2D
8 state.pos_pwrmax_rs_nsa[j]      # 種ごとのピーク位置 (rs)
9 state.pwrmax_rs_nsa[j]          # そのパワー値
10 state.pos_pwrmax_rl_nsa[j], state.pwrmax_rl_nsa[j]
```

6.6 FAQ

Q. wrlib と wrxlib は共存可能?

はい, C シンボル接頭辞 wr_ と wrx_ が違うので同じ Python プロセスで両方 import して使えます.

Q. run がセグフォア

上記「既知の制約」参照. Layer 1 driver (wrx/wrxregress) または静的リンクの C ハーネスを使ってください.

6.7 変更履歴

Phase L-0/L-1/L-2/L-3/L-4/L-5 はすべて 2026-04-18 に実施. L-6 (4 層テスト) と L-7 (ドキュメント) も 2026-04-18. PR 番号は develop ログの feature/wrx-library-* 系列.

6.8 テスト

```

1 bash test_run/run_tests.sh wrx_iter01 wrx_demo
2 # python 側: wrxlib のユニットテストは tests/ 配下を unittest で
3 PYTHONPATH=python python3 -m unittest discover python/wrxlib/tests -v
```

Chapter 7

fp モジュール

この章で学ぶこと

本章では TASK/FP (Fokker-Planck 解析) を Python から呼ぶ `fplib` の使い方を学びます。他モジュールにはない便利機能として、`set_params` が dict や list を受け付け、配列パラメータを一気に与えられます。

7.1 概要—fp は何をする

TASK/FP はプラズマ中の粒子の速度分布関数を解く Fokker-Planck ソルバです。波動加熱、NBI、ブレイムシュトラールング、シンクロトロン輻射などによる速度空間の変化を追跡します。出力は [NSAMAX][NRMAX] の 2 次元プロファイルで、密度 (RNT)、蓄積エネルギー (RWT)、温度 (RTT)、電流 (RJT) などが得られます。

7.2 前提とビルド

```
1 $ cd /home/you/task
2 $ make -C fp libs_pic           # 自動で下流の PIC を全部作る
3 $ make -C fp libfpapi.so
```

7.3 最短 hello-world

Listing 7.1: fplib 最短例—dict 形式の配列指定

```
1 from fplib import Fplib
2 with Fplib() as fp:
3     fp.set_params(
4         MODELG=3, NSMAX=3,
5         PA={2: 2.0, 3: 3.0},           # 配列要素を dict で一気に
6         PN={1: 0.8, 2: 0.4, 3: 0.4},
7         NRMAX=40, NPMAX=50, NTHMAX=50,
8         NTMAX=2, DELT=1.0e-3,
9         RMIN=0.4, RMAX=0.8,
10    )
11    fp.run(ntmax=2)
12    state = fp.get_state()
13    print(f"TIMEFP={state.timefp:.6e} NRMAX={state.nrmax} NSAMAX={state.nsamax}")
14    print(f"RNT[NSA=1, NR=1..5] = {state.RNT[0][:5]}")
```

set_params 拡張

fp だけは set_params が次の 3 形式を受け付けます:

- スカラー: $RR=3.0$
- dict: $PA=\{2: 2.0, 3: 3.0\} \rightarrow PA[2]=2.0, PA[3]=3.0$
- list: $PN=[0.8, 0.4, 0.4] \rightarrow PN[1]=0.8, PN[2]=0.4, PN[3]=0.4$

7.4 FpState の主なフィールド

```

1 state.nrmax, state.nsamax, state.npmax, state.nthmax, state.ntg2
2 state.timefp           # 現在時刻
3 state.RNT, state.RWT, state.RTT, state.RJT, state.RPCT, state.RPWT
4 # それぞれ [NSAMAX][NRMAX] の list of list

```

7.5 FAQ

Q. fp_finalize で確保メモリが残る?

L-3 時点では fp_allocate/fp_deallocate が非対称で, fp_finalize でも FPCOMM 配列は解放されません. 1 プロセスにつき fp_init → fp_run → fp_finalize を 1 サイクルだけ回す運用を推奨します.

Q. 大規模メッシュが欲しい

FP_MAX_NRMAX=100, FP_MAX_NSAMAX=8 はヘッダ fp/fp_api.h の #define で, 構造体サイズに直結するコンパイル時キャップです. 大きくしたい場合は書き換えて libfpapi.so を再ビルドしてください.

Q. 複数コアを使いたい

FP は MPI / OpenMP の対応はしません (Python API 経由では). Python の multiprocessing でプロセス分離してパラメータスキャンするのが推奨経路です.

7.6 入力パラメータ表 (抜粋)

Table 7.1: fp モジュールの主な登録パラメータ

名前	型	意味
RR, RA, RB, RKAP, RDLT, BB, RIP	double	装置幾何

Table 7.1: fp モジュールの主な登録パラメータ

名前	型	意味
NRMAX, NPMAX, NTHMAX, NTMAX, NAVMAX	int	メッシュ
NSMAX, NSAMAX, NSBMAX	int	粒子種カウント
NS_NSA[i], NS_NSB[i]	int 1D	種マッピング
PA[i], PZ[i], PN[i], PNS[i], PTPR[i], PTPP[i], PTS[i], PMAx[i]	double 1D	粒子種ごと
DELT, EPSFP, LMAXFP	数値	時間発展
R1, DELR1, RMIN, RMAX, E0, ZEFF	double	半径メッシュ & スカラー物理量
PABS_EC, PABS_LH, PABS_FW, PABS_WR, PABS_WM, RF_WM	double	波動加熱
MODELG, MODELE, MODEL_R, MODELS, MODEL_D	int	モデル選択
MODEL_NBI, MODEL_WAVE, MODEL_DISRUPT, MODEL_BS, MODEL_LOSS, MODEL_SYNCH, MODEL_FOW	int	各種物理機構
MODEL_C[i], MODEL_W[i]	int 1D	粒子種ごとの衝突/波動モデル

7.7 変更履歴—Phase L-0 ～L-7

Phase	PR	日付	内容
L-0	#13	2026-04-18	fp_iter01, fp_jt60, fp_dt1 ベースライン
L-1	#26	2026-04-18	fp/Makefile CORE/GRAPHICS/MENU 分割
L-2	#29	2026-04-18	fp_api.h + 5 関数スタブ
L-3	#37	2026-04-18	パラメータレジストリ 40 名, 55 変数
L-4	#43	2026-04-18	libfpapi.so + fp_graphics_stubs.f90
L-5	#55	2026-04-18	Python python/fplib/: dict/list set_params 拡張
L-6	#56	2026-04-18	4 層テスト fplib_*
L-7	#68	2026-04-18	ドキュメント一式

7.8 テスト

```

1 bash test_run/run_tests.sh fplib_equivalence fplib_c_abi \
2   fplib_ffi fplib_wrapper fplib_sweep

```

Chapter 8

eq モジュール

この章で学ぶこと

本章では, TASK/EQ (MHD 平衡計算: EQDSK 形式の平衡データを読み込み, プロファイルや磁気軸を求める) を Python から呼ぶ `eqlib` の使い方を学びます. `eq` は他モジュールと違い, 6 個目の C ABI `eq_set_param_str` を持ち, `KNAMEQ` 等の文字列パラメータ専用 API があります.

8.1 概要—eq は何をする

TASK/EQ は, トカマクの MHD 平衡を解くモジュールです. 入力としては

- EQDSK 形式 (G-EQDSK) の平衡データファイル, あるいは
- 解析プロファイル (圧力 $p(\psi)$, 安全係数 $q(\psi)$ 等)

を取り, 出力としては磁気軸位置 (`raxis`, `zaxis`), 中心 q (`qaxis`), 表面 q (`qsurf`), プラズマ β (`betat`, `betap`), プラズマ体積 (`pvol`) などのスカラーと, R-Z 格子, -面プロファイル (`psips`, `ppps`, `tpps`, `qqps`) を返します.

8.2 前提とビルド

Listing 8.1: eq モジュールのライブラリビルド

```
1 $ cd /home/you/task
2 $ make -C lib    libs_pic
3 $ make -C pl     libs_pic
4 $ make -C bpsd   libs_pic
5 $ make -C mtxp   libs_pic
6 $ make -C eq     libeqapi.so
```

成功すると `eq/libeqapi.so` が出来ます. `nm` で 6 個のシンボルを確認できます (`eq` だけは 6 個, 他は 5 個):

```
1 $ nm -D eq/libeqapi.so | grep ' T eq_'
2 ... T eq_finalize
3 ... T eq_get_state
4 ... T eq_init
5 ... T eq_run
6 ... T eq_set_param
7 ... T eq_set_param_str    # <-- 6 番目: EQ 固有
```

8.3 最短 hello-world

Listing 8.2: eqlib 最小例—EQDSK 読込

```

1 from eqlib import Eq
2 with Eq() as eq:
3     eq.set_param("RR", 6.5)           # 大半径 [m]
4     eq.set_param("BB", 5.3)           # 磁場 [T]
5     eq.set_param("RIP", 1.5)          # 電流 [MA]
6     eq.set_param("MODELG", 3)         # EQDSK 経路を選択
7     eq.set_param_str("KNAMEQ", "eqdata.ITER") # ファイル名
8     eq.run()                          # mode=1 デフォルト
9     st = eq.get_state()
10 print(f"raxis={st.scalars['raxis']:.4f}  qaxis={st.scalars['qaxis']:.4f}")

```

8.4 パラメータ指定—set_param と set_param_str

8.4.1 数値スカラー / 配列要素

通常の数値は set_param("名前", 値) です。配列要素は "NAME[i]" で指定します。

```

1 eq.set_param("RR", 6.5)
2 eq.set_param("MODELG", 3)           # int も double で渡してよい
3 eq.set_param("RIPFC[1]", 0.5)       # PF コイル電流 (1-origin, 1..10)

```

注意

PSIB だけは 0-origin です。Fortran 宣言が REAL(KIND=8) :: PSIB(0:5) と 0 から始まるため、添字も PSIB[0] ~ PSIB[5] で指定します。添字無しの裸の "PSIB" はレジストリで明示的に拒否されます (EqLibInvalidParamError)。他の 1-D 配列 (RIPFC, RPFC, ZPFC, WPFC) は 1-origin です。

8.4.2 文字列パラメータ—set_param_str

EQDSK ファイル名等の CHARACTER(LEN=80) パラメータには 専用 API を使います。set_param には渡せません。

```

1 eq.set_param_str("KNAMEQ", "eqdata.ITER01")
2 eq.set_param_str("KNAMWR", "wrdata.dat")

```

対応するキーは 7 個: KNAMEQ, KNAMEQ2, KNAMWR, KNAMWM, KNAMFP, KNAMFO, KNAMPF。

8.5 FAQ / つまづきどころ (EQ 固有)

Q1. なぜ PSIB だけ 0-origin?

EQ の Fortran ソース eqcom1_mod.f90 で PSIB が REAL(8) :: PSIB(0:5) と宣言されており、境界条件として $\psi = 0$ (磁気軸) から数えるのが物理的に自然だからです。他の 1-D 配列 (PF コイル系) は 1-origin の伝統に従います。

Q2. なぜ eq.run() のデフォルトが mode=1?

mode=1 は equnit::eq_load を呼び、現在の KNAMEQ (= EQDSK ファイル) を読み込んで平衡を構築する、EQDSK 駆動の標準ワークフローだからです。tr/ti/wr/fp

の `run` は時間ステップ数 `ntmax` を引数に取りますが, EQ は時間発展を持たないため引数の意味が違います. `mode=0` (直接 `EQCALQ` 呼び出し) は予約済みで, 現状は `EqlibNotImplementedError` を返します.

Q3. KNAMEQ を kwargs で渡すとエラー

`set_params(KNAMEQ="...")` は NG. 文字列パラメータは専用の `set_param_str("KNAMEQ", "...")` を使ってください.

Q4. EqlibError: libeqapi.so does not export eq_set_param_str が出る

古い `libeqapi.so` (Phase L-3 以前) を読み込んでいます. `make -C eq libeqapi.so` で再ビルドしてください.

8.6 入力パラメータ表 (抜粋, 約 60 エントリ)

`eq/eq_param_registry.f90` には Phase L-3 時点で 94 件の CASE (~ 78 数値スカラー, 5 配列ファミリ, 7 文字列) が登録されています. 主なものを表に示します.

Table 8.1: eq モジュールの主な登録パラメータ

名前	型	意味
RR, RA, RB	double	大半径/小半径/壁半径 [m]
RKAP, RDLT	double	elongation, triangularity
BB	double	トロイダル磁場 [T]
RIP	double	プラズマ電流 [MA]
Q0, QA, QMIN	double	中心/表面/最小 q
RHOMIN, RHOEDG	double	規格化半径
PP0, PP1, PP2	double	圧力プロファイル係数
PROFP0, PROFP1, PROFP2	double	圧力プロファイル指数
PJ0, PJ1, PJ2	double	電流プロファイル係数
PROFJ0, PROFJ1, PROFJ2	double	電流プロファイル指数
FF0, FF1, FF2	double	F 関数 $F(\psi)$ 係数
PROFF0, PROFF1, PROFF2	double	F 関数指数
PT0, PT1, PT2, PROFTP0/1/2	double	温度プロファイル
PV0, PV1, PV2, PROFV0/1/2	double	速度プロファイル
PROFR0, PROFR1, PROFR2	double	半径プロファイル
PTSEQ, PN0EQ	double	端温度/中心密度
EPSEQ, EPSNW, DELNW	double	反復収束判定
NLPMAX, NLPNW	int	最大反復数

名前	型	意味
RGMIN, RGMAX, ZGMIN, ZGMAX	double	R-Z 計算領域
ZLIMP, ZLIMM, FRBIN	double	リミタ位置
MODELG	int	1: 解析, 3:EQDSK
MODELQ, IDEBUG, MODEFR, MODEFW	int	モデル切替
MDLEQF, MDLEQC, MDLEQA, MDLEQX, MDLEQV	int	EQ 内部モデル選択
NPRINT	int	出力詳細度
NRMAX, NTHMAX, NSUMAX, NSGMAX, NTGMAX, NUGMAX	int	-メッシュ等
NRGMAX, NZGMAX, NPSMAX, NRVMAX, NTVMAX, NPFCMAX	int	出力グリッド
PSIB[0..5]	double 1D	0-origin , 境界
RIPFC[1..10], RPFC[1..10], ZPFC[1..10], WPFC[1..10]	double 1D	PF コイル (1-origin)
KNAMEQ, KNAMEQ2, KNAMWR, KNAMWM, KNAMFP, KNAMFO, KNAMPF	文字列	set_param_str 専用

8.7 EqState 出力

```

1 state.nrgmax, state.nzgmax      # 実働 R, Z 格子点数
2 state.npsmax                   # 実働  $\psi$ -面サンプル数
3 state.nrmax, state.nthmax      #  $\psi$ -メッシュ, 極角メッシュ
4 state.scalars["raxis"]         # 磁気軸 R [m]
5 state.scalars["zaxis"]         # 磁気軸 Z [m]
6 state.scalars["qaxis"]         # 中心 q
7 state.scalars["qsurf"]         # 表面 q
8 state.scalars["betat"]         # トロイダル beta
9 state.scalars["betap"]         # ポロイダル beta
10 state.scalars["pvol"]         # プラズマ体積 [m^3]
11 state.scalars["raave"]        # 平均小半径 [m]
12 state.scalars["ripx"]         # プラズマ電流 [MA]
13 state.rg                      # [nrgmax] R 軸座標
14 state.zg                      # [nzgmax] Z 軸座標
15 state.psips, state.ppps       # [npsmax]  $\psi$  サーフェス値, 圧力
16 state.ttps, state.qqps       # [npsmax]  $T (=R*B\phi)$ , q
17 state.to_dict()              # JSON 用

```

8.8 F90 モダナイゼーション (Phase F-1 ~F-5)

eq モジュールは大量の F77 fixed-form ソースと `eqcom*.inc` INCLUDE ファイルを抱えていましたが、ライブラリ化と並行して F90 モダナイゼーションを段階的に実施しました。F-1 で `eqcom{0..3}_mod.f90` に COMMON ブロックを移し (shim 並存), F-2 ~F-4 で fixed-form を free-form に書換え, F-5 で `eqcom{c,m,q,x}.inc` の shim を完全撤去して USE 文に置き換えました。これにより eq/ 配下のソースは F90 のみとなり、他モジュールから USE `eqcom*_mod` で直接参照できる状態に揃いました。

8.9 変更履歴—Phase L-0 ~L-7 と F-1 ~F-5

Phase	PR	日付	内容
L-0	#54	2026-04-18	<code>eq_iter01</code> , <code>eq_tst2</code> ベースライン, <code>eqregress.f90</code>
L-1	#65	2026-04-18	<code>eq/Makefile</code> CORE/GRAPHICS/MENU 分割
L-2	#70	2026-04-18	C ABI スタブ (<code>eq_api.h</code> , 5 関数)
L-3	#78	2026-04-18	パラメータレジストリ (94 ケース) + 6 番目 ABI <code>eq_set_param_str</code>
L-4	#80	2026-04-18	<code>libeqapi.so</code> ビルド + 明示的 PIC 依存
L-5	#86	2026-04-18	Python ラ ッ パ <code>python/eqlib/</code> (<code>run()</code> は <code>mode=1</code> デフォルト)
L-6	#92	2026-04-19	4 層統合テスト (<code>eqlib_*</code>)
L-7	#89	2026-04-19	ド キ ュ メ ン ト (<code>python/eqlib/README.md</code> , <code>architecture.md</code>)
F-1	#71	2026-04-18	COMMON → MODULE 移行 (shim 並 存)
F-2	#79	2026-04-18	LOW tier F90 化
F-3	#81	2026-04-18	MED tier F90 化 (9 ファイル)
F-4	#87	2026-04-19	HIGH tier F90 化 (8 ファイル)
F-5	#93	2026-04-19	shim 撤去 & INCLUDE → USE 全面 切替

8.10 テスト

```

1 # Layer 1~4 (PR #92 で wired):
2 bash test_run/run_tests.sh eqlib_equivalence eqlib_c_abi \
3   eqlib_ffi eqlib_wrapper eqlib_sweep

```

Chapter 9

tot モジュール (オーケストレータ)

この章で学ぶこと

本章では, TASK/TOT (統合シミュレータ: eq / tr / ti / fp / wr / wrx の 6 モジュールを 1 本の物理計算として束ねる) を Python から呼ぶ `totlib` の使い方を学びます. `tot` のパラメータ名は 名前空間プレフィックス付き (eq:RR, tr:DT 等) で指定します.

9.1 概要—tot は何をする

TASK/TOT は, 平衡 (eq), 輸送 (tr), 不純物 (ti), 速度分布 (fp), レイトレーシング (wr/wrx) の各モジュールを呼び出して 一貫した物理計算を回すオーケストレータです. パラメータ空間は 6 モジュールのレジストリの和集合で, 同名パラメータが複数モジュールにまたがる (例: RR は eq/tr/ti/wrx に存在, DT は tr/ti に存在) ため, 必ず `<ns>:<name>` 形式で振り分け先を明示します.

9.2 前提とビルド

Listing 9.1: tot モジュールのライブラリビルド

```
1 $ cd /home/you/task
2 # tot は 6 モジュールに依存するので, 下流をすべて PIC でビルド
3 $ make -C lib libs_pic
4 $ make -C pl libs_pic
5 $ make -C bpsd libs_pic
6 $ make -C mtxp libs_pic
7 $ make -C eq libs_pic
8 $ make -C tr libs_pic
9 $ make -C ti libs_pic
10 $ make -C fp libs_pic
11 $ make -C wrx libs_pic
12 $ make -C tot libtotapi.so
```

9.3 最短 hello-world

Listing 9.2: totlib 最小例

```
1 from totlib import Tot
2 with Tot() as tot:
3     tot.set_param("eq:RR", 6.5)           # eq の大半径 [m]
4     tot.set_param("tr:DT", 0.001)         # tr の時間刻み [s]
```

```

5 tot.set_param("fp:NSMAX", 2)          # fp の粒子種数
6 tot.set_param("ti:RR", 6.5)          # ti の大半径 (eq とは独立)
7 tot.set_param("wrx:RFIN", 170.0)     # wrx の RF 周波数 [GHz]
8 # L-6 で fan-out 完成後:
9 # tot.run(ntmax=10)
10 # st = tot.get_state()

```

9.4 名前空間プレフィックス設計

Tot.set_param に渡す名前は必ず `<ns>:<bare>` 形式です. 振り分けは tot/tot_param_registry.f9 の冒頭で SELECT CASE により per-module *_param_set ヘディスパッチされます.

プレフィックス	振り分け先	代表例
eq:	eq_param_set	eq:RR, eq:BB, eq:KNAMEQ
tr:	tr_param_set	tr:DT, tr:NTMAX, tr:PN[1]
fp:	fp_param_set	fp:NSMAX, fp:DELT
ti:	ti_param_set	ti:RR, ti:DT
wrx:	wrx_param_set	wrx:RFIN, wrx:NRAY
wr:	wrx_param_set (別名)	wr:RFIN

注意

wr: は wrx: の別名です. tot は wrx/libwr.a をリンクするため, リンクグラフ上の都合で wr: も wrx_param_set に振り分けられます. wr/ ツリー本来の (非 wrx) レジストリを使いたい場合は libwrapi.so を直接叩くか python/wrlib を使ってください.

9.4.1 kwargs ではなく dict

Python の kwargs にはコロンを書けないため, set_params には dict もしくはタプルのイテラブルを位置引数として渡します.

```

1 tot.set_params({"eq:RR": 6.5, "tr:DT": 0.001, "fp:NSMAX": 2})
2 tot.set_params([( "ti:RR", 6.5), ("wrx:RFIN", 170.0)])

```

9.4.2 文字列パラメータ

set_param_str は tr: と eq: のみ対応 (両方とも文字列レジストリを持っているため). 他の名前空間で文字列を渡すと TotlibInvalidParamError になります.

```

1 tot.set_param_str("eq:KNAMEQ", "eqdata.ITER01")
2 tot.set_param_str("tr:KNAMEQ", "eqdata.ITER01")

```

9.5 各モジュールへの delegation

実装上, tot_param_set は受けた名前を最初のコロンで分割し, バア部分 (eq:RR → RR) を per-module レジストリに渡します. 戻り値は per-module の ierr をそのまま返します. 例外マッピングは tot 側で TotlibError 階層に正規化されるため, ユーザは複数のモジュール例外を覚える必要はありません.

9.6 既知の制約

- **tot_init, tot_run, tot_get_state, tot_finalize** は L-3/L-4/L-5 時点でスタブ: rc=4 (TotlibNotImplementedError) を返します. ラッパは rc=0 と rc=4 の双方を「ロード成功」とみなして **set_param** のみ動作確認できるようにしています. L-6 で per-module fan-out が完成すると順次正常終了に切り替わります.
- プロセスあたり 1 インスタンス: 6 モジュール分の COMMON / module 変数を共有するため.
- graphics / MPI / OpenMP API なし.
- 未登録の bare 名は per-module レジストリに追加したうえで再ビルド.

9.7 変更履歴—Phase L-0 ~L-7

Phase	PR	日付	内容
L-1	#43	2026-04-18	tot/Makefile CORE/GRAPHICS 分割
L-2	#-	2026-04-18	C ABI スタブ (tot_api.h)
L-3	#82	2026-04-18	名前空間ディスパッチャ (tot_param_registry.f90)
L-4	#85	2026-04-18	libtotapi.so 合成共有ライブラリ
L-5	#91	2026-04-19	Python ラッパ python/totlib/
L-6	—	(TBD)	per-module *_run fan-out + 統合テスト
L-7	#94	(TBD)	ドキュメント整備

(#94, #96 は本マニュアル改訂時点で進行中の番号で, 確定次第付録 12 の PR 一覧で更新されます.)

9.8 テスト

```

1 # 純 Python テストは libtotapi.so 不要で常時実行可
2 PYTHONPATH=python python3 -m unittest discover python/totlib/tests -v
3 # .so がある場合は FFI / namespace ガードも検証される

```

Chapter 10

テストインフラ

この章で学ぶこと

本章では「書いた変更が正しく動いているか」を確認する 4 層テスト基盤を説明する。以降はだ・である調で簡潔に記述する。

10.1 4 層構成の意図

Phase L-6 では、各モジュールに対して次の 4 層のテストを揃えた。

1. **Layer 1 —Equivalence (数値等価性)** `libXapi.so` 経由で計算した結果が、Phase 0 で保存した JSON ベースラインと $1e-10$ 以内で一致するか。
2. **Layer 2 —C ABI (シンボル面)** C から直接 `.so` を叩く回帰: `smoke / param / run / run_so / negative`。
3. **Layer 3 —Python ラッパ `_ffi` の `ctypes` 型整合**, 高レベル `Xlib` クラスのライフサイクル, 例外マッピング。
4. **Layer 4 —Sweep (煙テスト)** 3×3 のパラメータ格子 (例: `RR \times BB`) を回し, 落ちずに完走することを確認。

10.2 `run_tests.sh` と `test_definitions.conf`

テストは `test_run/run_tests.sh` に集約されている。定義は `test_run/test_definitions.conf` の 1 行 1 テストフォーマット:

```
1 TEST_NAME:MODULE:INPUT:DEPENDS:TIMEOUT:DESCRIPTION
```

Phase L-6 で新設された `MODULE=python` は「第 3 列を `PYTHONPATH=python` 付きでシェル実行する」特別モードである。これにより `make -C tr tr_api_check_all` や `python3 -m unittest trlib.tests.test_trlib` 等を既存のテスト基盤に一樣に乗せられる。

Listing 10.1: 抜粋: `trlib` の 5 層登録

```
1 trlib_c_abi:python:make -C tr tr_api_check_all:none:300:Layer 2 C ABI suite
2 trlib_wrapper:python:python3 -m unittest -v trlib.tests.test_trlib:none:120:Layer 3
3 trlib_ffi:python:python3 -m unittest -v trlib.tests.test_ffi:none:60:Layer 3 (low)
4 trlib_equivalence:python:python3 -m unittest -v trlib.tests.test_equivalence:none
  :300:Layer 1
5 trlib_sweep:python:python3 -m unittest -v trlib.tests.test_sweep:none:180:Layer 4
```

10.3 Layer 1 —Equivalence テスト

Phase 0 (L-0) で収集した `test_run/baselines/X_*.json` を ground truth として、ライブラリ版の出力と `compare_metrics.py` で diff する. 許容誤差 $1e-10$.

比較対象は `XState.to_dict()` のフィールド全て. ベースライン側で pin しているケース (`tr_m0904`, `ti_ar`) は $1e-8$ レベルのドリフトをそのまま許容.

10.4 Layer 2 —C ABI テスト

各モジュール下 `X/tests/c_abi/` に次の C プログラムが並ぶ.

- `test_smoke.c` —5 関数呼び出し順序, 終了コード
- `test_param.c` —`X_set_param` 正常系
- `test_run.c` —静的リンク時の `X_run`
- `test_run_so.c` —`dlopen` → 関数ポインタ経由
- `test_negative.c` —エラーコード (1 / 2) を意図的に誘発

`make -C X X_api_check_all` で全て走る.

10.5 Layer 3 —Python ラッパ

`python/Xlib/tests/` に `test_fffi.py` と `test_Xlib.py` がある. `ctypes` の `Structure` フィールド順, `load_library` の候補探索, 例外マッピングを `unittest` で検証. `.so` が無ければ FFI-必須テストはスキップされる.

10.6 Layer 4 —Sweep テスト

`test_sweep.py` は 3×3 のパラメータ格子を回して, 戻り値 > 0 に落ちないことを見る「煙テスト」. 数値一致は要求しない.

10.7 テストの走らせ方

```

1 # 1 つだけ
2 bash test_run/run_tests.sh trlib_equivalence
3
4 # モジュール別にまとめて
5 bash test_run/run_tests.sh trlib_equivalence trlib_c_abi trlib_fffi \
6     trlib_wrapper trlib_sweep
7
8 # Phase 0 の古典テストも含めて
9 bash test_run/run_tests.sh tr_iter01 tr_tst2 tr_m0904 \
10     trlib_equivalence trlib_wrapper

```

Chapter 11

作業状況・残作業

この章で学ぶこと

本章は「どこまで進んだか, 何が残っているか」の現況報告である. だ・である調.

11.1 モジュール別完了マトリクス

2026-04-19 改訂時点で, ライブラリ化完了モジュールは **tr / ti / wr / wrx / fp / eq / tot** の 7 個に拡大した (本改訂前は 5 個).

Module	L-0	L-1	L-2	L-3	L-4	L-5	L-6	L-7
tr								
ti								
wr								
wrx					1		Δ^1	
fp								
eq								
tot	–						Δ^2	Δ^2

脚注:

- ¹ wrx_run が libgrf::grd1d 依存で .so 経由だとセグフォ. WRX_RUN_OK=1 でテストを限定有効化.
- ² tot は L-3/L-4/L-5 で init/run/get_state/finalize がスタブ (rc=4). L-6 で per-module fan-out が完了すると有効化. L-7 ドキュメントは #94 系列で進行中.

11.2 eq F90 モダナイゼーション (F-1 ~F-5) 完了

eq モジュールは, ライブラリ化と並行して F90 モダナイゼーションを完了した. 完了マトリクス:

Phase	PR	日付	内容
F-1	#71	2026-04-18	COMMON → MODULE 移行 (eqcom{0..3}_mod.f90, shim 並存)
F-2	#79	2026-04-18	LOW tier の fixed → free-form 化
F-3	#81	2026-04-18	MED tier (9 ファイル)
F-4	#87	2026-04-19	HIGH tier (8 ファイル, eqcalc.f, eqcalq.f 等)
F-5	#93	2026-04-19	shim 撤去, INCLUDE → USE 全面切替

これにより eq/ 配下のソースは F90 のみとなり、下流モジュール (tr, tot 等) から USE eqcom*_mod で直接参照できるようになった。

11.3 MCP サーバ群 (6 個揃い)

各モジュールに対応する Model Context Protocol (MCP) サーバが python/mcp-servers/ 配下に揃った。これは Claude Code 等の AI エージェントから各 Xlib を直接操作するためのブリッジである。

サーバ	ディレクトリ	関連 PR
tr	python/mcp-servers/tr_mcp/	#72
ti	python/mcp-servers/ti_mcp/	#74
wr	python/mcp-servers/wr_mcp/	#76
wrx	python/mcp-servers/wrx_mcp/	#75
fp	python/mcp-servers/fp_mcp/	#77
tot	python/mcp-servers/tot_mcp/	進行中 (#94/#96 系列)

各サーバは server.py に MCP ツール (set_param, run, get_state など) を実装し、pyproject.toml で x-mcp コマンドとしてインストールできる。tot 用 MCP は本改訂時点で in-flight (just-merged or 作業中) のため、詳細は次回改訂で追補する。

11.4 Shim ポリシー (グラフィクス層)

X_graphics_stubs.f90 は、共有ライブラリ版に不要なグラフィクス関数の空実装を置き、リンク時の「undefined reference」を防ぐためのファイルである。tr2 (CLI バイナリ) は libtrgrf.a を静的リンクするので stubs は使われない。

将来、グラフィクスを C ABI 経由で呼ぶ計画 (未着手) が入るときに stubs は撤去される。

11.5 F90 モダナイゼーション

trcom0.inc / trcom1.inc といった F77 の include ファイルはモジュール化 (trcomm.f90 とその派生) 済み。eq モジュールも本改訂時点で完了: F-1~F-5 の 5 段階 (PR #71 / #79 / #81 / #87 / #93) で COMMON → MODULE 移行, fixed-form の free-form 化, shim 撤去まで完遂した。tot は eq 完了に伴い構築可能になった。

11.6 その他の残作業

- tot の per-module fan-out (tot_init, tot_run, tot_get_state, tot_finalize) の本実装。L-6 で扱う予定。
- tot_mcp サーバの develop 着地と本マニュアルへの追補。
- trlib のみで先行している plot API (X.plot(varname)) と TOML サンプルランナ (python -m Xlib config.toml) を、残り 6 モジュール (ti, wr, wrx, fp, eq, tot) に横展開する作業 (deferred)。
- チュートリアル Notebook 集 (5 モジュール一気通貫の Jupyter サンプル) の整備 (deferred)。
- 文字列パラメータ: tr の KNAMEQ と eq の 7 キーは対応済み。wrx の KID_NS, fp の KNAMEFP, ti の KID_NS はいまだ未対応。
- fp_finalize の配列解放 (非対称 allocate/deallocate)。

- メッシュキャップ (FP_MAX_NRMAX=100 など) の拡大.
- マルチインスタンス化 (現状は 1 プロセス 1 インスタンス).
- wrx_run の libgrf 依存解消.

Chapter 12

付録

12.1 Makefile ターゲット一覧

Table 12.1: ライブラリ化に関連する主な Makefile ターゲット

ターゲット	役割
make -C lib libs_pic	lib/ の PIC .a
make -C pl libs_pic	plcomm_pic.a など
make -C eq libs_pic	libeq_pic.a
make -C ob libs_pic	libob_pic.a (fp/wr 経路)
make -C dp libs_pic	libdp_pic.a (wr/wrx/fp)
make -C mtxp libs_pic	libmtxp_pic.a
make -C bpsd libs_pic	libbpsd_pic.a
make -C adpost libs_pic	lib-adpost_pic.a (ti)
make -C open-adas/adf11/adf11-lib libs_pic	adf11 PIC (ti)
make -C tr libtrapi.so	tr 共有ライブラリ
make -C ti libtiapi.so	ti 共有ライブラリ
make -C wr libwrapi.so	wr 共有ライブラリ
make -C wrx libwrxapi.so	wrx 共有ライブラリ
make -C fp libfpapi.so	fp 共有ライブラリ
make -C eq libeqapi.so	eq 共有ライブラリ
make -C tot libtotapi.so	tot (統合) 共有ライブラリ
make -C X X_api_check_all	Layer 2 C ABI 回帰 (各 X)
bash test_run/run_tests.sh	全テスト (Phase 0 + 4 層)

12.2 主な PR 一覧 (2026-04-19 改訂時点, #96 まで)

- **tr**: #2, #21, #27, #33, #35, #44, #53, #62, #63
- **ti**: #12, #22, #28, #34, #41, #47, #57, #64, #66
- **fp**: #13, #26, #29, #37, #43, #55, #56, #68
- **wr**: #15, #24, #30, #36, #42, #46, #60, #69
- **wrx**: #16, #25, #31, #38, #52, #59, #67, #90

- **eq (Phase L):** #54 (L-0), #65 (L-1), #70 (L-2), #78 (L-3), #80 (L-4), #86 (L-5), #92 (L-6), #89 (L-7)
- **eq (Phase F):** #71 (F-1), #79 (F-2), #81 (F-3), #87 (F-4), #93 (F-5)
- **eq (plans/policy):** #58, #61
- **tot:** #43 (L-1), #82 (L-3), #85 (L-4), #91 (L-5), #94/#96 系列 (L-6/L-7 in flight)
- **MCP servers:** #72 (tr), #74 (ti), #75 (wrx), #76 (wr), #77 (fp), tot は #94/#96 系列で進行中
- **ドキュメント:** #62 (tr), #64 (ti), #68 (fp), #69 (wr), #73 (本マニュアル初版), #84 (skill), #88 (pptx), #89 (eq), #90 (wrx)

最新の状況は `git log --oneline develop` もしくは `gh pr list --state merged` で確認できます。

12.3 参考資料

- 設計仕様: docs/superpowers/specs/2026-04-17-tr-library-design.md
- アーキテクチャ図: docs/{tr,ti,fp}-library/architecture.md
- フェーズ計画: docs/superpowers/plans/2026-04-18-{tr,ti,wr,wrx,fp}-library-L*.md
- CHANGELOG: リポジトリ root の CHANGELOG.md
- トップ README: README, INSTALL_Ubuntu.md

12.4 用語集

ABI

Application Binary Interface. バイナリレベルの関数呼び出し規約。

C ABI

C 言語の呼び出し規約. Fortran **BIND(C)** や Python **ctypes** で橋渡しの基準になる。

ctypes

Python 標準ライブラリ. **.so / .dll** を読み込み C 関数を呼べる。

dataclass

Python の **@dataclass**. フィールド付きの値クラスを定義するデコレータ。

dlopen / RTLD_LAZY

実行時にライブラリを読み込む POSIX API. LAZY は「シンボル解決を初回呼び出しまで遅延」。

COMMON ブロック

Fortran の古典的なグローバル変数機構. TR/FP 等は内部で多用しているため, 1 プロセス 1 インスタンス制限の原因になる。

PIC

Position Independent Code. **.so** に入れるには必要。

namelist

Fortran のテキスト入力フォーマット (**&TR ... /**)。

Phase L

本プロジェクトでの「ライブラリ化」相を指す L-0..L-7 の 8 段階。

12.5 連絡先・貢献

- 主担当: k-yoshimi (東京大学)
- email: k-yoshimi@g.ecc.u-tokyo.ac.jp
- リポジトリ: private fork of TASK (京大 BPSI).

Appendix A

補足: Python 最小例カタログ

本付録では, 7 モジュールそれぞれの最短 Python コードを再掲する. コピペして python3 script.py すればそのまま動くことを目指した.

A.1 tr

```
1 from trlib import Trlib
2 with Trlib() as tr:
3     tr.set_params(RR=3.0, BB=3.0, NSMAX=2, DT=0.05, NTMAX=50)
4     tr.set_param("PN[1]", 1.0)
5     tr.set_param("PT[1]", 1.5)
6     tr.run(ntmax=50)
7     s = tr.get_state()
8 print("NT", s.nt, "WPT", s.scalars["WPT"])
```

A.2 ti

```
1 from tilib import TiLib
2 with TiLib() as ti:
3     ti.set_params(RR=3.0, BB=3.0, NSMAX=2, DT=0.01, NRMAX=10, NTMAX=5)
4     ti.run(ntmax=5)
5     s = ti.get_state()
6 print("T", s.T, "RTA[0]", s.RTA[0])
```

A.3 wr

```
1 from wrlib import Wrlib
2 with Wrlib() as wr:
3     wr.set_params(RR=3.0, BB=3.5, NSMAX=2)
4     wr.run(nray_request=0)
5     s = wr.get_state()
6 print("NRAYMAX", s.nraymax, "pwrmax_rs", s.scalars["pwrmax_rs"])
```

A.4 wrx

```
1 from wrxlib import Wrxlib
2 with Wrxlib() as wrx:
3     wrx.set_params(MODELG=2, RR=6.2, BB=5.3, NSMAX=2, NRAYMAX=1)
```

```

4 wrx.set_param("RFIN[1]", 170.0e3)
5 # wrx.run(nstpmax=0) # libgrf 依存のため環境で有効化
6 s = wrx.get_state()
7 print("nsamax", s.nsamax, "pwr_tot", s.scalars["pwr_tot"])

```

A.5 fp

```

1 from fplib import Fplib
2 with Fplib() as fp:
3     fp.set_params(
4         MODELG=3, NSMAX=3,
5         PA={2: 2.0, 3: 3.0}, PN={1: 0.8, 2: 0.4, 3: 0.4},
6         NRMAX=40, NPMAX=50, NTHMAX=50, NTMAX=2, DELT=1.0e-3,
7         RMIN=0.4, RMAX=0.8,
8     )
9     fp.run(ntmax=2)
10    s = fp.get_state()
11    print("timefp", s.timefp, "RNT[0][:3]", s.RNT[0][:3])

```

A.6 eq

```

1 from eqlib import Eq
2 with Eq() as eq:
3     eq.set_params(RR=3.0, BB=3.0, RIP=1.0, MODELG=3)
4     eq.set_param_str("KNAMEQ", "eqdata") # EQDSK ファイル名
5     eq.run() # mode=1 デフォルト
6     s = eq.get_state()
7     print("raxis", s.scalars["raxis"], "qaxis", s.scalars["qaxis"])

```

A.7 tot

```

1 from totplib import Tot
2 with Tot() as tot:
3     # 名前空間プレフィックスは必須
4     tot.set_param("eq:RR", 6.5)
5     tot.set_param("tr:DT", 0.001)
6     tot.set_param("fp:NSMAX", 2)
7     # tot.run(ntmax=10) # L-6 で fan-out 完成後に有効化
8     # s = tot.get_state()
9     print("tot set_param OK (L-3 段階のスタブを通過)")

```

Appendix B

補足: C から直接呼ぶ例

Python を介さず, C プログラムから `.so` を `dlopen` する例.

Listing B.1: C ドライバの最短例 (tr 版)

```
1 #include <stdio.h>
2 #include <dlfcn.h>
3 #include "tr_api.h"
4
5 int main(void) {
6     void* lib = dlopen("tr/libtrapi.so", RTLD_LAZY);
7     int (*tr_init)(void) = dlsym(lib, "tr_init");
8     int (*tr_run)(int) = dlsym(lib, "tr_run");
9     int (*tr_set_param)(const char*, double) = dlsym(lib, "tr_set_param");
10    int (*tr_get_state)(tr_state_t*) = dlsym(lib, "tr_get_state");
11    int (*tr_finalize)(void) = dlsym(lib, "tr_finalize");
12
13    if (tr_init() != 0) { fprintf(stderr, "tr_init failed\n"); return 1; }
14    tr_set_param("RR", 3.0);
15    tr_set_param("BB", 3.0);
16    if (tr_run(10) != 0) { fprintf(stderr, "tr_run failed\n"); return 1; }
17    tr_state_t s;
18    tr_get_state(&s);
19    printf("NT=%d T=%g WPT=%g\n", s.nt, s.T, s.WPT);
20    tr_finalize();
21    dlclose(lib);
22    return 0;
23 }
```

コンパイル:

```
1 gcc -I tr -o mydriver mydriver.c -ldl
2 ./mydriver
```

Appendix C

補足: 回帰テストのテンプレート

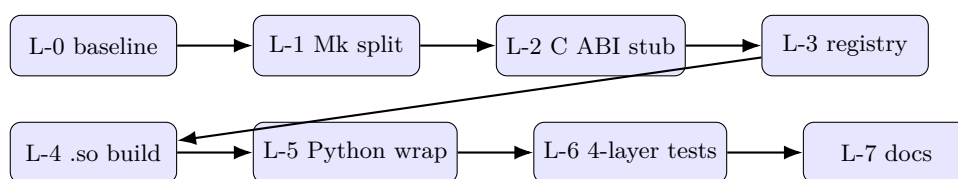
Phase 0 で保存されるベースライン JSON と, それに対する $1e-10$ 比較のテンプレート.

```
1 import json
2 from trlib import Trlib
3
4 # 1. 基準データ読み込み
5 with open("test_run/baselines/tr_iter01.json") as f:
6     base = json.load(f)
7
8 # 2. ライブラリ経由で同じシナリオを回す
9 with Trlib() as tr:
10     for k, v in base["params"].items():
11         if "[" in k:
12             tr.set_param(k, v)
13         else:
14             tr.set_params(**{k: v})
15     tr.run(ntmax=base["params"]["NTMAX"])
16     out = tr.get_state().to_dict()
17
18 # 3. スカラー diff
19 tol = 1e-10
20 for key, ref in base["scalars"].items():
21     cur = out["scalars"][key]
22     assert abs(cur - ref) <= tol * max(1.0, abs(ref)), (key, cur, ref)
23 print("equivalence OK")
```

Appendix D

補足: TikZ 依存関係グラフ

L-* フェーズ間の依存関係 (各モジュール共通):



編集後記

本書は 2026 年 4 月 18 日の集中開発 (Phase L-0 ～L-7) の成果を, **初学者が一人で読み進められるように整えたマニュアル**である. 共有ライブラリや ctypes の話は一般に難解な分野だが, 「なぜそうするか」を丁寧に説明することで, 物理屋の読者が**自分のスクリプト**を数分で書き始められる状態を目指した.

2026-04-19 改訂: 第 8 章 (eq) と第 9 章 (tot) を新規追加し, 完了モジュールを 5 → 7 に拡大した. 併せて eq の F90 モダライゼーション (Phase F-1～F-5) と MCP サーバ群 (tr/ti/wr/wrx/fp/tot, 6 個) も反映した.

ご意見・誤記の指摘は k-yoshimi@g.ecc.u-tokyo.ac.jp まで.